

Autoencoder-Based Latent Representations for Visual Reinforcement Learning in CarRacing-v3

Andrea Catino

Bachelor's Degree Programme in Applied Computer Science and Artificial Intelligence (ACSAI)

Course: AI-LAB – Computer Vision, Signal Processing, and Natural Language Processing

Department of Computer Science, Sapienza University of Rome

Matricola: 2154867

catino.2154867@studenti.uniroma1.it

Abstract

Visual reinforcement learning requires an agent to learn perception and control from high-dimensional image observations. This project investigates whether an offline visual representation learned by a convolutional autoencoder can improve the learning behaviour of Proximal Policy Optimization (PPO) in CarRacing-v3. The baseline trains PPO end-to-end from 96×96 RGB frames with a convolutional policy. The proposed pipeline first trains an autoencoder on 18,293 frames collected during environment interaction, freezes its encoder, and trains an MLP policy from 128-dimensional latent observations. The experimental protocol compares both methods with identical learning rate, interaction budget, evaluation tracks, and three predefined training seeds. Performance is measured through deterministic evaluation reward, final-window reward, area under the learning curve, threshold crossing time, and variability across seeds. Across three training seeds, latent PPO improves best evaluation reward by 10.6%, final-window reward by 49.2%, and normalized learning-curve AUC by 50.2%. It reaches a reward of 700 with 45.8% fewer timesteps on average, although both methods remain variable across seeds. The contribution is a reproducible comparison that separates representation learning from control and critically analyses the benefits and limitations of reconstruction-based features for visual control.

Keywords: visual reinforcement learning; representation learning; autoencoder; proximal policy optimization; sample efficiency.

1 Introduction

Reinforcement learning (RL) agents learn control policies by interacting with an environment and maximizing cumulative reward. When observations are images, the agent must also discover which visual information is relevant for action selection. This coupling between perception and control makes visual RL substantially more demanding than learning from compact state vectors. The success of deep Q-networks demonstrated that convolutional features can be learned directly from pixels and reward signals [4], but end-to-end visual RL remains data-intensive and sensitive to optimization choices.

This project studies continuous visual control in Gymna-

sium CarRacing-v3. Each observation contains 27,648 pixel values, while the action contains only steering, throttle, and brake. The central research question is:

Does a latent representation learned by a convolutional autoencoder improve PPO sample efficiency, stability, or final performance compared with PPO trained directly from raw pixels?

The motivation is to decouple representation learning from policy learning. Instead of requiring PPO to discover visual features only through reward, an autoencoder first learns to compress environment frames by minimizing reconstruction error. Its encoder is then frozen and used as a deterministic observation transform. This produces a clean comparison between an end-to-end raw-pixel policy and a policy trained from a fixed, compact representation.

The personal contribution is not a new RL algorithm. It is the design and implementation of a complete experimental pipeline: collection of a visual dataset from policy interaction, training and qualitative analysis of a convolutional autoencoder, integration of its encoder through a Gymnasium observation wrapper, controlled PPO training, deterministic evaluation, and multi-seed statistical comparison.

The remainder of the paper reviews related work, presents both pipelines and the autoencoder, defines the reproducible experimental protocol, reports the multi-seed results, and discusses their implications and limitations.

2 Related Work

2.1 Visual Reinforcement Learning

Deep visual RL commonly learns a convolutional feature extractor jointly with a policy or value function. The DQN work of Mnih et al. established an influential end-to-end approach for learning control from pixels [4]. However, visual observations introduce high dimensionality and nuisance variation, and reward provides only an indirect supervision signal for perception. Later work has shown that visual augmentation can substantially improve pixel-based RL by regularizing learned representations [2]. The present project deliberately retains a simple unaugmented PPO baseline so that the effect of a frozen autoencoder can be isolated.

2.2 Policy Optimization

PPO is an on-policy actor-critic method that alternates environment interaction with several epochs of optimization on collected rollouts [6]. Its clipped surrogate objective limits the incentive for large policy updates:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)], \quad (1)$$

where $r_t(\theta)$ is the probability ratio between the updated and previous policies and $\hat{A}_t$ is an advantage estimate. PPO is attractive for this project because the same algorithm supports both image observations through a CNN policy and vector observations through an MLP policy. Nevertheless, clipping does not guarantee monotonic improvement, so periodic evaluation and best-model selection remain necessary.

2.3 Autoencoders and Latent Control

Autoencoders learn a low-dimensional code by reconstructing their input. In visual control, a compressed representation can reduce the dimensionality presented to the policy and separate visual learning from reward-driven control. SAC+AE studied reconstruction as an auxiliary objective for model-free control from images and showed that encoder training and gradient flow strongly affect stability [7]. CURL instead learns visual features through a contrastive objective and demonstrates that representation choice can improve sample efficiency in pixel-based RL [3]. The method in this project is intentionally simpler than both: the autoencoder is pretrained offline, then frozen before PPO training.

This simplicity also exposes an important limitation. Pixel reconstruction does not explicitly distinguish control-relevant information from irrelevant visual detail. A representation may obtain low MSE while smoothing road boundaries or encoding the simulator HUD. The project therefore treats the autoencoder as a hypothesis to test, not as an automatically superior feature extractor.

3 Methodology

3.1 Environment

The environment is `CarRacing-v3`, a `Box2D` continuous-control task [1]. The observation is an RGB image with shape $96 \times 96 \times 3$ and `uint8` values. The action is a three-dimensional continuous vector for steering, throttle, and brake. Experiments use `lap_complete_percent=0.95` and `domain_randomize=False`. The default observation includes the simulator HUD at the bottom of the frame. Keeping it in both conditions preserves comparability, although it is a methodological limitation.

3.2 Raw-Pixel Baseline

The baseline follows the upper path in Fig. 1. `Stable-Baselines3` [5] transposes observations to channel-first format and trains PPO with `CnnPolicy`. Its convolutional extractor, actor, and value function are optimized jointly from PPO

Table 1: Autoencoder dataset summary.

Property	Value
Source	CarRacing policy interaction
Samples	18,293 RGB frames
Input format	$96 \times 96 \times 3$, PNG
Training split	90%
Validation split	10%
Normalization	Pixel values in $[0, 1]$

rollouts. No external representation pretraining or image augmentation is used.

3.3 Frame Dataset

Frames were sampled during earlier PPO interaction using random intervals to reduce temporal redundancy. Sampling was delayed after environment resets to avoid over-representing nearly identical starting frames. The resulting local dataset contains 18,293 PNG images. Images are loaded as RGB, converted to `float32`, normalized from $[0, 255]$ to $[0, 1]$, and transposed to $3 \times 96 \times 96$. A fixed random split assigns 90% of frames to training and 10% to validation.

3.4 Convolutional Autoencoder

The encoder applies four stride-two convolutions with channel dimensions $3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$. Spatial resolution is reduced from 96 to 6, producing a $256 \times 6 \times 6$ feature map. A linear layer maps the flattened 9,216 values to a 128-dimensional latent vector. The decoder mirrors this structure with a linear layer and four transposed convolutions. A sigmoid output matches normalized RGB values.

For input x and reconstruction $\hat{x}$, training minimizes mean squared error:

$$\mathcal{L}_{\text{AE}} = \frac{1}{CHW} \sum_{c,h,w} (x_{c,h,w} - \hat{x}_{c,h,w})^2. \quad (2)$$

The model was trained with Adam, batch size 128, and learning rate 10^{-4} for 40 epochs in two consecutive 20-epoch sessions. Before the second session, the model weights were retained while the Adam optimizer was reinitialized. The final training and validation losses are 0.000581 and 0.000659, respectively. The model has approximately 3.75 million parameters.

3.5 Latent Observation Pipeline

The lower path in Fig. 1 is implemented through a `Gymnasium` observation wrapper. For each observation, the wrapper reproduces autoencoder preprocessing, executes the encoder without gradient tracking, and returns a 128-dimensional `float32` vector. Encoder parameters are frozen and the model remains in evaluation mode. PPO then uses `MlpPolicy`. Consequently, the RL optimizer cannot adapt the visual representation to the control task. This restriction is intentional because it isolates the effect of offline reconstruction pretraining.

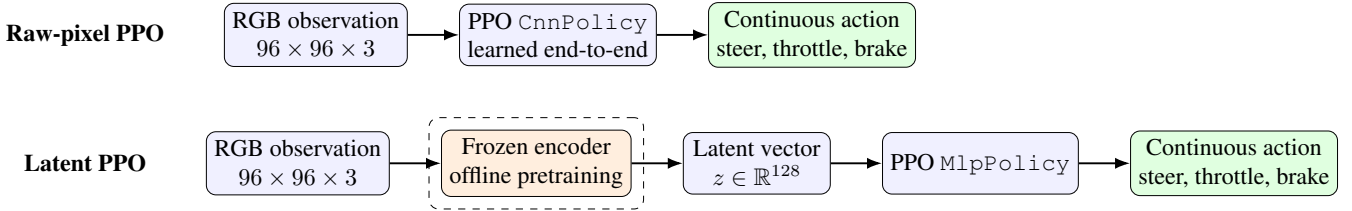


Figure 1: Controlled comparison between the raw-pixel baseline and the proposed latent-observation pipeline. Both agents interact with the same environment and use PPO; only the visual representation and policy architecture differ.

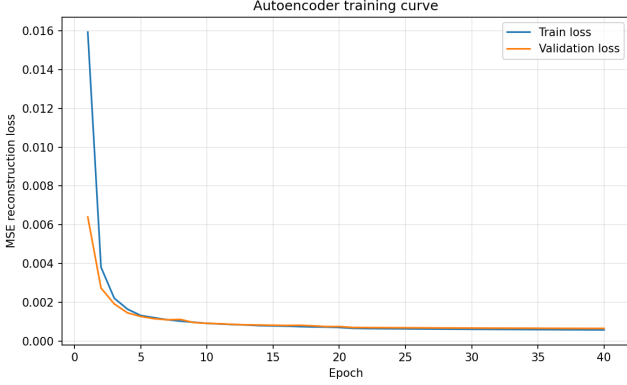


Figure 2: Autoencoder training and validation MSE over 40 epochs.

Table 2: Main controlled PPO configuration.

Parameter	Value
Algorithm	PPO
Training seeds	42, 123, 456
Evaluation base seed	10,000
Parallel environments	2
Interaction budget	2,000,000 steps
Learning rate	10^{-4}
Evaluation interval	20,000 steps
Episodes per evaluation	5
Raw policy	CnnPolicy
Latent policy	MlpPolicy
Latent dimension	128

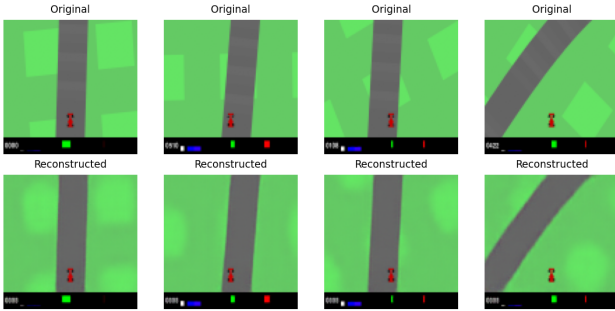


Figure 3: Original CarRacing observations (top) and autoencoder reconstructions (bottom). Global road geometry and car position are retained, while boundaries and textures are smoothed by the pixel-wise MSE objective.

4 Experimental Setup

4.1 Software and Hardware

The implementation uses Python 3.11, PyTorch, Gymnasium 1.2.3, and Stable-Baselines3 2.8.0. Experiments run on an NVIDIA GeForce RTX 3080 Ti and an AMD Ryzen 7 5800X CPU. Raw-pixel PPO uses CUDA, whereas latent PPO and its frozen encoder use the CPU in the current configuration.

4.2 Controlled Multi-Seed Protocol

The main benchmark consists of three independent training seeds, $\{42, 123, 456\}$, for each method. Both methods use the same PPO learning rate, interaction budget, number of parallel environments, and deterministic evaluation protocol. Training and evaluation use distinct seeds. The evaluation

seed is fixed across methods so corresponding policies are tested on the same procedurally generated tracks.

The callback parameter is set to 10,000 calls. Because two vectorized environments advance at each callback step, this corresponds to 20,000 aggregate environment timesteps between evaluations. Each evaluation averages five deterministic episodes. The best checkpoint is selected by evaluation reward, and the final checkpoint is also retained.

4.3 Metrics

The following metrics are defined before observing the controlled benchmark:

- best mean deterministic evaluation reward;
- timestep of the best evaluation;
- mean reward over the final five evaluations;
- normalized area under the evaluation learning curve (AUC);
- first timestep at which mean reward reaches 700;
- mean and standard deviation of scalar metrics across training seeds.

The final-window mean is preferred over a single last evaluation because CarRacing returns are noisy and PPO learning is non-monotonic. Wall-clock time is not a primary metric because the two policies use different devices and runs may be scheduled at different times.

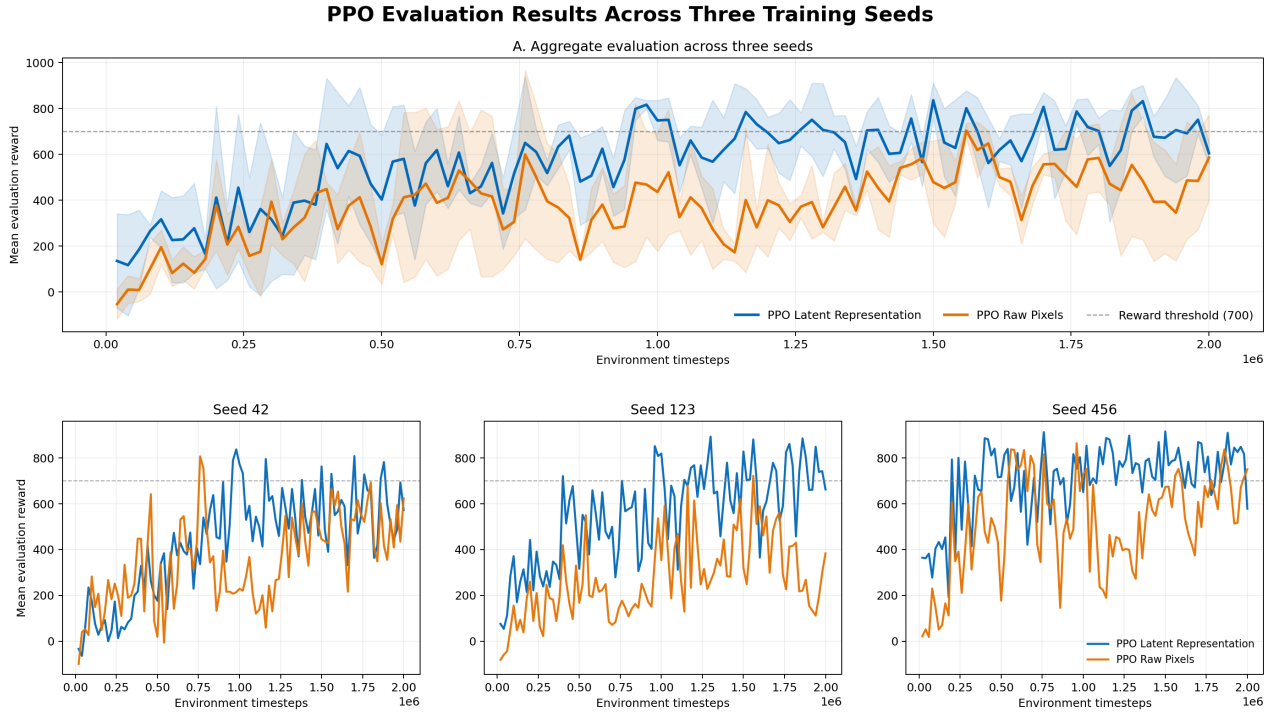


Figure 4: Deterministic PPO evaluation results. The upper panel shows the mean reward across three training seeds, with shaded regions indicating one between-seed standard deviation. The lower panels compare raw and latent PPO within each matched seed. Each point averages five evaluation episodes.

Table 3: Controlled results across three training seeds. Values are mean $\pm$ standard deviation across seeds. For steps to reward 700, lower is better.

Metric	Raw PPO	Latent PPO	Relative difference
Best evaluation reward	798.1 $\pm$ 71.7	882.4 $\pm$ 40.5	+10.6%
Final-window reward	459.2 $\pm$ 211.3	685.1 $\pm$ 127.4	+49.2%
Normalized AUC	377.8 $\pm$ 120.0	567.5 $\pm$ 134.5	+50.2%
Steps to reward 700	960k $\pm$ 529k	520k $\pm$ 394k	-45.8%

4.4 Controlled Results

Table 3 reports the aggregate metrics across the three training seeds. Latent PPO performs better on all four predefined aggregate metrics. Its mean best evaluation reward increases from 798.1 to 882.4, while the final-window mean increases from 459.2 to 685.1. The larger improvement in final-window reward indicates that the latent policies maintain stronger performance near the end of training rather than only producing an isolated high-reward checkpoint.

Figure 4 shows that the latent mean curve is higher through most of training and crosses the reward-700 threshold earlier on average. The matched-seed panels also expose substantial PPO variance. Latent PPO reaches the threshold earlier for seeds 123 and 456, whereas raw PPO reaches it earlier for seed 42. Nevertheless, latent PPO obtains higher best reward, final-window reward, and normalized AUC in all three matched seeds.

4.5 Critical Discussion

The results support three cautious observations. First, a frozen 128-dimensional representation retains sufficient in-

formation for PPO to learn effective driving behaviour despite compressing 27,648 pixel values. The consistent improvements in AUC and final-window reward suggest that offline visual pretraining simplified policy learning in this experimental setting. Second, the benefit is not explained only by a fortunate checkpoint: latent PPO achieves higher best reward, final-window reward, and normalized AUC for every matched training seed.

Third, the frozen representation does not eliminate RL instability. The shaded regions in Fig. 4 remain wide, and threshold-crossing time has large standard deviation for both methods. With only three seeds, the experiment provides descriptive rather than strong inferential statistical evidence. The results therefore support the latent pipeline within the tested *CarRacing-v3* protocol, but they do not establish that reconstruction pretraining is generally superior to end-to-end visual RL.

The comparison also changes the policy architecture from *CnnPolicy* to *MlpPolicy*, because the observation spaces differ. This is intrinsic to the research question but means the measured effect combines representation pretraining, dimensionality reduction, and a smaller downstream policy input. It should not be interpreted as an isolated ablation of encoder initialization.

5 Limitations

The experiment has several limitations. Three seeds improve reliability but remain a small sample for a high-variance RL benchmark, and no significance test is meaningful at this sample size. Evaluation averages only five episodes at each checkpoint. The frame dataset was collected from policy in-

teraction and may under-represent rare recovery states. The HUD is included in all images, and the autoencoder may allocate capacity to it. MSE encourages smooth reconstructions and may suppress sharp road boundaries. The encoder is frozen, preventing task-specific adaptation, and only one latent dimension and one main learning rate are evaluated. Finally, conclusions are limited to `CarRacing-v3` and may not generalize to other visual-control tasks.

6 Conclusion

This project implements a reproducible comparison between end-to-end raw-pixel PPO and PPO trained from a frozen autoencoder representation in `CarRacing-v3`. The pipeline includes policy-based frame collection, a convolutional autoencoder, a latent observation wrapper, controlled PPO training, deterministic evaluation, and multi-seed aggregation. Under the tested protocol, latent PPO improves best reward by 10.6%, final-window reward by 49.2%, and normalized AUC by 50.2%, while reaching reward 700 with 45.8% fewer timesteps on average. The advantage appears across all three matched seeds for best reward, final-window reward, and AUC, although both methods retain substantial variance.

These findings indicate that a compact frozen representation can improve sample efficiency and overall performance in this setting. They do not imply that low reconstruction error guarantees generally useful control features. The main contribution is therefore the complete reproducible pipeline and controlled critical comparison, rather than a new RL algorithm.

Future work should evaluate cropped observations without the HUD, alternative latent dimensions, visual augmentation, robustness to perturbations, and task-aware encoder fine-tuning.

Acknowledgments

The project uses the open-source Gymnasium, Stable-Baselines3, and PyTorch ecosystems.

References

- [1] Farama Foundation. Gymnasium car racing documentation. https://gymnasium.farama.org/environments/box2d/car_racing/, 2026. Accessed: 2026-06-09.
- [2] Ilya Kostrikov, Denis Yarats, and Rob Fergus. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In *International Conference on Learning Representations*, 2021.
- [3] Michael Laskin, Aravind Srinivas, and Pieter Abbeel. CURL: Contrastive unsupervised representations for reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5639–5650, 2020.
- [4] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare,

Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

- [5] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
- [6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [7] Denis Yarats, Amy Zhang, Ilya Kostrikov, Brandon Amos, Joelle Pineau, and Rob Fergus. Improving sample efficiency in model-free reinforcement learning from images. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 10674–10681, 2021.